

Does the Combined use of Class and Sequence Diagrams Improve the Source Code Comprehension? Results from a Controlled Experiment

Giuseppe Scanniello
DMI, University of Basilicata
Potenza, Italy
giuseppe.scanniello@unibas.it

Carmine Gravino and Genoveffa Tortora
University of Salerno
Fisciano, Salerno, Italy
{gravino, tortora}@unisa.it

ABSTRACT

We present the results of a controlled experiment aimed to investigate whether the source code comprehension increases when participants are provided with UML class and sequence diagrams produced in the software design phase. The experiment has been conducted with Master students in Computer Science at the University of Salerno. The data analysis shows that the participants significantly better comprehend source code when it is added with class and sequence diagrams together.

Categories and Subject Descriptors: D.2.0 [Software Engineering]: General

General Terms: Experimentation, Measurement

Keywords: Comprehension, Controlled Experiment, UML

1. INTRODUCTION

Nowadays the documentation of object oriented software systems contains several UML (Unified Modeling Language) diagrams [16]. The wide diffusion of the UML is due to the number of visual notations it offers to write system blueprints, including conceptual (e.g., business processes and system functions) and concrete items (e.g., programming language statements, database schema, and reusable software components) [10].

The assessment of the benefits deriving from the use of the UML in all the phases of the software life cycle is relevant for the software engineering community as shown by the number of empirical studies in terms of controlled experiments and case studies available in the literature [4]. Although a number of studies have been conducted on the UML, only a few of them have been carried out to assess whether the use of UML models produced in the design phase improves source code comprehension [4].

In this paper, we present the results of a controlled experiment conducted with last year Master students in Computer Science at the University of Salerno. We analyze whether the

participants achieved an improved comprehension of source code when it is added with UML class and sequence diagrams produced in the design phase. The control group is represented by the participants who performed comprehension tasks with source code alone, while the treatment group is constituted by the participants who performed the comprehension tasks class and sequence diagrams together.

The remainder of the paper is organized as follows. The design of the experiment is presented in Section 2, while we present and discuss the achieved results in Section 3. The threats that may affect the validity of the results are highlighted in Section 4. In Section 5, we discuss related work. Final remarks and future work conclude.

2. CONTROLLED EXPERIMENT

The planning and the design of the experiment is reported in this section. We followed the template for experimentation in software engineering suggested in [21]. For replication purposes, we made available on the web¹ an experimental package and the raw data.

2.1 Definition

Applying the Goal Question Metric (GQM) paradigm [2], the goal of the experiment can be defined as follows: *Analyze the use of UML class and sequence diagrams produced in the design phase for the purpose of evaluating them with respect to their support in the comprehension of source code from the point of view of the researcher, in the context of students in Computer Science, and from the point of view of the project manager, in the context of junior programmers.*

2.2 Planning

2.2.1 Context

Participants were last year students enrolled to a Master program in Computer Science at the University of Salerno. They can be considered not far from novice software engineers or junior programmers [5].

The participants to the experiment (UniSa from here on) were all graduate students with basic software engineering knowledge. They had knowledge of requirements engineering, high and low level design of object oriented software

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

EESSMOD'12, October 01 - 05 2012, Innsbruck, Austria
Copyright 2012 ACM 978-1-4503-1811-2/12/10 ...\$15.00.

¹www.scienzefn.unisa.it/scanniello/SC_UML_Exp1/

systems based on the UML, software development, and software maintenance. The students were asked to accomplish the experiment as an optional activity of an Advanced Software Engineering course of the Master program in Computer Science at the University of Salerno.

There were 16 participants, who were not graded on the comprehension results achieved in the experiment. We asked the participants to perform the tasks based on the experimental objects individually and in a professional way to get one point more to the final mark.

We used two systems for managing: (i) the sales of a music shop (i.e., *Music Shop*) and (ii) the reservation of theater tickets (i.e., *Theater Ticket Reservation*). The systems Music Shop and Theater Ticket Reservation were developed by students of the Master program in Computer Science at the University of Basilicata as the laboratory activity of an Advanced Object Oriented Programming course. These systems never underwent maintenance operations.

The experimental objects used were selected within these systems. The selection was guided keeping in mind a trade-off between the complexity and the relevance of the implemented functionality. The effort to perform comprehension tasks (about 1 hour) on these objects was another criterion used. For Music Shop, we selected a chunk of 463 LOCs (S1 from here on). For the second system, we selected a chunk of 378 LOCs (S2 from here on). As far as S1 is concerned, the class diagram contained: 6 classes, 27 attributes, and 45 methods; while the sequence diagram: 1 actor, 5 objects, and 11 messages. For S2, the class diagram contained: 5 classes, 22 attributes, and 36 methods; while the sequence diagram: 1 actor, 5 objects, and 9 messages. S1 and S2 were small enough to fit the time constraints of the experiment though realistic for small maintenance operations that a novice software engineers or junior programmer perform within a software company [20].

2.2.2 Hypothesis Formulation and Selected Variables

We defined and tested the following null hypothesis:

Hn0 The presence of UML class and sequence diagrams *does not significantly improve* the comprehension of source code.

The alternative hypothesis (i.e., Ha0) can be easily derived because it admits a positive effect of UML class and sequence diagrams on source code comprehension.

To test the hypothesis above, we considered the independent variable (also named main factor) *Method*. It is a nominal variable that can assume two values: *NO_Mo*(source code alone) and *Mo* (source code added with UML diagrams).

To quantitatively evaluate the comprehension achieved by the participants on the source code, we asked them to fill out a comprehension questionnaire for each experimental object. These questionnaires consisted of 15 multiple choice questions, each admitting the same number of possible answers with only one correct. Then, we defined and used the following dependent variable:

Table 1: Experiment design

	Group A	Group B	Group C	Group D
Run 1	S1, Mo	S1, NO_Mo	S2, Mo	S2, NO_Mo
Run 2	S2, NO_Mo	S2, Mo	S1, NO_Mo	S1, Mo

Comprehension: the number of correct answers provided by the participant.

2.2.3 Experiment Design

We adopted the within-participant counterbalanced experimental design. It is shown in Table 1: each participant worked on S1 and S2 using either Mo or NO_Mo. The students were randomly assigned to each group: A, B, C, and D. Each group contained 4 students.

We also analyzed the effect of:

System. Differences in the experimental objects (e.g., complexity and domain) could affect the comprehension of the participants.

Order of Method. The order in which the participants performed the laboratory runs may bias the results.

2.2.4 Execution of the Experiment

Pilot. A pilot experiment was conducted to evaluate possible issues related to the experimental material. The participants to the pilot were 2 Bachelor students in Computer Science at the University of Basilicata. The results indicated that 2 hours were sufficient to accomplish the experiment. The pilot results allow identifying minor issues in the experimental material. These issues were properly fixed before the experiment.

Experiment Execution. The experiments were organized in three phases. In the first phase the participants attended an introductory lesson on how to execute the comprehension tasks. The participants were informed of the pedagogical purpose of the experiment. Details on the experimental hypotheses were not given.

The second and third phases were sequentially performed in the same day. In the second phase, the participants were asked to accomplish the comprehension tasks according to the adopted design in the two subsequent laboratory runs. Each participant was provided with the following material:

- Handouts of the introductory presentation.
- For each object, we provided the participants with the source code. Depending on the task, we also furnished the UML diagrams. A printout of the associated comprehension questionnaire was also given. The material for the second laboratory run was given to the participants only when they accomplished the first run.
- A post-experiment survey questionnaire used to gain enough insight to strengthen and explain the results of each experiment. The post-experiment survey questionnaire was filled out at the end of the second task and collected by the supervisors together with the material of the second task.

- Some sheets of paper and a pencil.

The post-experiment survey questionnaire was defined to get indications about the overall quality of the provided material, the perceived usefulness of the models, the clarity of the experimental objects, and the goals of the experiment. The used post-experiment survey questionnaire is shown in Table 2.

2.3 Analysis Procedure

We used the non-parametric Wilcoxon test to assess the effect of method (i.e., accept the H_{n0}). This test was also used in all the cases paired analyses were required (e.g., to study the effect of System). In case of unpaired analysis, we chose the Mann-Whitney U exact test. We employed these non parametric tests due to the sample size and (in some cases) the non-normality of the distributions.

Statistical tests check the presence of statistical significant differences, but they do not provide any information about the magnitude of such a difference. We then used the point-biserial correlation r because is the best way to compute the magnitude of the difference when a Wilcoxon test is used [14]. The r value is computed a $\sqrt{Z^2/N}$, where Z is returned by the Wilcoxon test and N is the sample size. In the empirical software engineering fields [18], the magnitude of the effect sizes measured using the point-biserial correlation is classified as follows: small (0 to 0.193), medium (0.193 to 0.456), and large (0.456 to 0.868).

Further, to analyze the probability that a statistical test will reject a null hypothesis when it is actually false, we analyzed the statistical power of the tests performed. A value close to 0.80 is considered as a standard for the adequacy [13].

To test the effect of Order of Method, we used a method similar to the one suggested by Briand *et al.* in [3]. Let be: $Diff(NO_Mo)$ the differences for the Comprehension values achieved by the participants, who (according to the experimental design) performed the tasks with NO_Mo first and then with Mo; $Diff(Mo)$ the differences for the Comprehension values achieved by the participants, who per-

formed the tasks with Mo first and then with NO_Mo. We applied the non-parametric Mann-Whitney U exact test to verify $Had: Diff(NO_Mo) > Diff(Mo)$.

For all the performed statistical tests, we decided to accept a probability of 5% of committing a Type-I-Error [21].

To investigate the effect of System, we applied the Bonferoni correction [8], [9]. In particular, we used two tests to analyze whether the effect of System is statistically significant. Then, the p-values have to be less than $\alpha_{cor} = \frac{0.050}{2} = 0.025$.

To graphically show the answers of the post-experiment survey questionnaire, we adopted boxplots. These are widely employed since they provide a quick visual representation to summarize data [9].

3. ANALYSIS AND RESULTS

Some descriptive statistics (i.e., median, mean, and standard deviation) on the Comprehension dependent variable are shown in Table 3. The statistics are grouped by Method and System. These descriptive statistics show that the participants achieved a better comprehension when they accomplished the task with the UML diagrams. Furthermore, the participants achieved nearly the same comprehension on S1 and S2 both with Mo and NO_Mo.

3.1 Influence of Method

The results of the Wilcoxon test revealed that H_{n0} can be rejected (p-value < 0.01). The effect size is large (i.e., 0.62) and the statistical power is 0.91. An average improvement of 14% was achieved when the participants accomplished the comprehension task with the class and sequence diagrams.

The results of a paired analysis is reported in Table 4. The results show that the number of participants that benefited from the UML diagrams in the source code comprehension (Mo > NO_Mo) were 13 out of 16, while 2 were those who obtained a better comprehension using source code alone (Mo < NO_Mo). One participant out of 16 achieved the same comprehension using Mo and NO_Mo (Mo = NO_Mo), respectively. For each participant, we also computed the difference between the comprehension values he/she achieved with Mo and that achieved with NO_Mo (i.e., Mo - NO_Mo). This value is positive in case the participant achieved a better comprehension with Mo, otherwise negative. The difference is zero, when the same comprehension values was achieved by the participant using Mo and NO_Mo. Some descriptive statistics on Mo - NO_Mo are reported in Table 4. In particular, the average value is 1.31, while the median is 1. The minimum and maximum are -3.00 and 4.00, respectively. The standard deviation is 1.66.

3.2 Effect of co-factors

System - The Mann-Whitney test revealed that there was no significant effect of System. When using Mo and NO_Mo, the p-values are 1 and 0.87, respectively.

Order of Method - the Mann-Whitney test revealed that the order in which the participants performed the comprehension tasks was not statistically significant. These results indicated that the participants did not get a significantly

Table 2: Post-experiment Survey Questionnaire

Id	Question	Answers
Q1	I had enough time to perform the tasks	(1-5)
Q2	The task objectives were perfectly clear to me	
Q3	The tasks I performed were perfectly clear to me	
Q4	Judge the difficulty of the task concerning the system Music Shop (i.e., S1)	(A-E)
Q5	Judge the difficulty of the task concerning the system Theatre Ticket Reservation (i.e., S2)	
Q6	Using the UML class and sequence diagrams the comprehension of a software system is enhanced	(1-5)
1 = strongly agree; 2 = agree; 3 neutral; 4 = disagree; 5 = strongly disagree		
A = very high; B = high; C = medium; D = low; E = very low		

Table 3: Descriptive Statistics

System	Observation	Mo					NO_Mo				
		Min	Max	Med	Mean	Std. Dev.	Min	Max	Med	Mean	Std. Dev.
All	32	10	15	14	13.06	1.53	9	14	12	11.75	1.29
S1	16	11	14	14	13.12	1.25	9	13	11.5	11.62	1.41
S2	16	10	15	13.5	13	1.85	10	14	12	11.88	1.25

Table 4: Analysis of the differences

Mo > NO_Mo	Mo < NO_Mo	Mo = NO_Mo	Descriptive statistics of Mo – NO_Mo				
			Min	Max	Med	Mean	Std. Dev.
13/16	2/16	1/16	-3.00	3.00	1.00	1.31	1.66

better comprehension of the source code when passing from the first task to the second one.

3.3 Further analysis

We also collected information on the time the participants spent to accomplish a comprehension task on source code. We then performed a further analysis on that information to complete the assessment on the affect of the UML class and sequence diagrams on the source code comprehension.

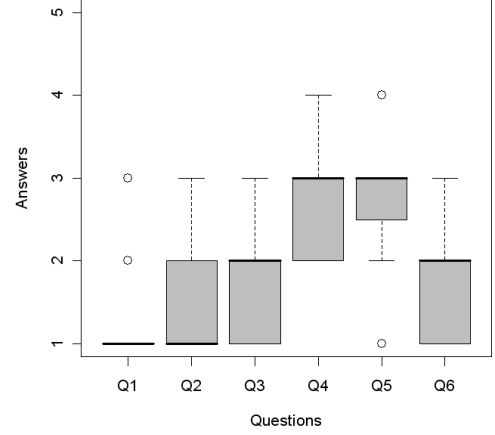
The Wilcoxon test revealed that there was no statistically significant difference in the completion time (i.e., the dependent variable) when using Mo and NO_Mo (p-value = 0.12). The hypothesis tested with this non parametric test was two-sided because we could not make any postulation on the effect of Method on completion time. Therefore, the combined use of UML class and sequence diagrams does not significantly affect the time the participants spent to comprehend source code. However, the descriptive statistics indicate that the participants on average spent more time when using Mo (mean = 30.06) with respect to NO_Mo (mean = 23). The median values are 27.5 for Mo and 20 for NO_Mo. Furthermore, the standard deviation for Mo is almost twice that of NO_Mo (i.e., 13.13 and 7.61, respectively).

The Mann-Whitney test revealed that there was not a significant effect of System on the observed results. The p-values are 0.53 and 0.79 when using Mo and NO_Mo, respectively. Furthermore, the effect of Order of Method was not statistically significant as well. The returned p-value was 0.37.

It is worth mentioning that the investigation on the completion time was not our primary goal here. This is because we confined the discussion on that dependent variable in this subsection.

3.4 The Results of the Post-experiment Survey Questionnaire

The boxplots reported in Figure 1 summarize the answers to the post-experiment survey questionnaire of UniSa. As the box of Q1 shows, the participants to experiment considered the time to conduct the experiment appropriate (the median is 1 and corresponds to strongly agree). They also clearly understood both the objectives and the tasks. The boxes of Q2 and Q3 show that the medians are 1 (strongly agree) and 2 (agree), respectively. A neutral judgment on the complexity of the experimental objects was given as the boxes of Q4 and Q5 show: both the medians are 3. As the box

**Figure 1: Boxplots of the answers**

of Q6 shows, all the participants found the use of the UML effective for the comprehension of source code (the median is 2 and corresponds to agree).

4. THREATS TO VALIDITY

In the following subsections the threats that could affect the validity of the controlled experiments are presented. We use the schema proposed in [21].

4.1 Internal Validity

Internal validity concerns the degree to which conclusions can be drawn about the causal effect of the independent variables on the dependent variables.

Interaction with selection. This threat has been mitigated because each group of participants worked on different objects with Mo or NO_Mo on two comprehension tasks. Further, the participants had similar experience with UML, software system modeling, and computer programming.

Maturation. Participants might have learned how to improve their comprehension performances passing from the first task to the second one. The data analysis revealed that the order in which the participants performed the tasks did not significantly affect source code comprehension.

Diffusion or imitation of treatments. This threat concerns the information exchanged among the participants. The participants were monitored by the experiment supervisors, who did not allowed the participants to communicate each other.

4.2 External Validity

The main issue of the external validity refers to the possibility of generalizing the results.

Interaction of selection and treatment. The use of students may affect the external validity [5], [6], [17]. Threats are related to the representativeness of the participants as compared with professionals. Replications with this kind of participants are needed to confirm or contradict the achieved results.

Interaction of setting and treatment. It concerns the experimental objects used. The software systems on which the experimental objects were selected have never undergone maintenance operations. This may affect the source code comprehension. Also, the size and complexity of the objects may affect the validity of the results. However, larger and more complex tasks could excessively overload the participants, thus biasing the results.

Using source code printout could have negatively affected the performance of the participants. It is worth noting that the use of a IDE should equally affect the comprehension of the participants both using and not using the diagrams.

4.3 Construct Validity

Construct validity concerns generalizing the results to the concepts behind the experiment. Some threats are related to the design of the experiments and to social factors.

Interaction of different treatments. The adopted design partially mitigated these threats.

Mono-method bias. Using one single dependent variable could bias the results. The adopted measure to quantitatively assess the comprehension is well known and widely used (e.g., [11], [15]).

Confounding constructs and level of construct. More levels than High and Low could be used in the classification of participants' ability. We are also aware that the use of a different measure to assess participants' ability could lead to different results. Also, the way to group the students into high and low experienced participants could represent another threat.

Evaluation apprehension. We mitigated this threat because the participants were not evaluated on the comprehension they achieved on the source code used in the experiment. The participants were not aware of the experimental hypotheses investigated.

Experimenters' expectations. We mitigated this threat formulating the questions of the comprehension questionnaires so conditioning their answers in favor of neither Mo nor NO_Mo. The post-experiment survey questionnaire was designed to capture the participants' perception on the experiment and the used material. The survey was intended to support and explain the quantitative results and was designed using standard approaches and scales [19].

4.4 Conclusion Validity

Conclusion validity concerns issues that may affect the ability of drawing a correct conclusion.

Reliability of measures. The used measure allowed us to assess in an objective way the comprehension level achieved by the participants on source code.

Random heterogeneity of participants. We drew fair samples and conducted our experiment with participants belonging to these samples. Another threat is related to the number of observations. Replications on a larger number of participants are required to confirm or contradict the results.

Fishing and the error rate. For UniSa the null hypothesis has been rejected with p-value less than 0.01 and 0.91 as the statistical power value.

5. RELATED WORK

We discuss the related literature concerning controlled experiments aimed at assessing the effect of using the UML in software maintenance and program comprehension. To get a deeper understanding on empirical evaluations on the models and forms used in the UML, a systematic literature review is available in [4].

Arisholm *et al.* [1] observe that the availability of documentation based on the UML may significantly improve the functional correctness of changes as well as the design quality when complex tasks have to be accomplished. Although this study and the one presented here have the same research goal (i.e., impact of UML documentation on software maintenance), a number of differences are present. The main difference is that we specifically focus on comprehension tasks, while the authors on modification tasks performed both on UML diagrams and source code.

Dzidek *et al.* [12] investigate on the costs and benefits in using the UML to maintain and evolve software systems. The authors conduct a controlled experiment with professional programmers. The results reveal that the use of the UML significantly impacts the functional correctness of the maintenance operations. Conversely, the UML use does not significantly affect the time to perform maintenance operations. The main difference with respect to our investigation is that the focus of the controlled experiment is not based on the comprehension of source code and the UML diagrams are different.

Gravino *et al.* [15] present a controlled experiment to assess whether UML models produced in the early phases of the development process (i.e., requirements elicitation and analysis phases) improve the comprehension of source code. The results reveal that the use of models does not significantly improve the comprehension of source code with respect to the use of source code alone. In some way, the study presented here and that highlighted in [15] go in the same direction: investigating whether or not the comprehensibility of the source code might improve when it is added with UML diagrams. The most remarkable difference between these papers is that here we consider UML diagrams produced in the design phase.

6. CONCLUSION AND FUTURE WORK

In this paper, we have presented the results of a controlled experiment to assess whether the comprehension of source code is affected when it is added with UML class and sequence diagrams produced in the design phase. The data analysis revealed that the participants benefited from the use of the UML diagrams. We used a controlled experiment because a number of confounding and uncontrollable factors could be present in real project settings. Controlled experiments are often conducted in the early steps of empirical investigations that take place over the years (e.g., [1], [7]).

Due to the results of the experiment, we plan to conduct industrial case studies in the future. This part of our research is still in progress and is the most challenging and complex. The results presented in the paper suggest the following future research directions:

1. It is possible that participants, who were provided with the UML diagrams, did not deeply analyze the code because they believed that these diagrams provided adequate information to accomplish a comprehension task on source code. This point is interesting from the researcher perspective and then worthwhile being investigated.
2. It could be interesting to investigate the motivation guiding the software engineers in trusting software models and how they select them to accomplish their tasks. It will be then worth analyzing how software engineers choose the information to perform comprehension tasks.

Acknowledgments

We wish to thank the Ilaria Bilancia and the Michela Continanza for their precious help in preparing the experimental material and in executing the experiments. We also thank the participants to the experiment.

7. REFERENCES

- [1] E. Arisholm, L. C. Briand, S. E. Hove, and Y. Labiche. The impact of UML documentation on software maintenance: An experimental evaluation. *IEEE Transactions on Software Engineering*, 32:365–381, 2006.
- [2] V. Basili, G. Caldiera, and D. H. Rombach. *The Goal Question Metric Paradigm, Encyclopedia of Software Engineering*. John Wiley and Sons, 1994.
- [3] L. C. Briand, Y. Labiche, M. Di Penta, and H. Yan-Bondoc. An experimental investigation of formality in UML-based development. *IEEE Transactions on Software Engineering*, 31(10):833–849, 2005.
- [4] D. Budgen, A. J. Burn, O. P. Brereton, B. A. Kitchenham, and R. Pretorius. Empirical evidence about the UML: a systematic literature review. *Software: Practice and Experience*, 41(4):363–392, 2011.
- [5] J. Carver, L. Jaccheri, S. Morasca, and F. Shull. Issues in using students in empirical studies in software engineering education. In *Proceedings of the 9th International Symposium on Software Metrics*, pages 239–, Washington, DC, USA, 2003. IEEE Computer Society.
- [6] M. Ciolkowski, D. Muthig, and J. Rech. Using academic courses for empirical validation of software development processes. *EUROMICRO Conference*, 0:354–361, 2004.
- [7] M. Colosimo, A. De Lucia, G. Scanniello, and G. Tortora. Evaluating legacy system migration technologies through empirical studies. *International Journal on Information and Software Technology*, 51(12):433–447, 2009.
- [8] W. J. Conover. *Practical Nonparametric Statistics*. Wiley, 3rd edition, 1998.
- [9] J. L. Devore and N. Farnum. *Applied Statistics for Engineers and Scientists*. Duxbury, 1999.
- [10] P. J. Dobing B. How uml is used. *Communications of the ACM*, 49(5):109–113, 2006.
- [11] J. Dolado, M. Harman, M. Otero, and L. Hu. An empirical investigation of the influence of a type of side effects on program comprehension. *IEEE Transactions on Software Engineering*, 29:665–670, 2003.
- [12] W. J. Dzidek, E. Arisholm, and L. C. Briand. A realistic empirical evaluation of the costs and benefits of UML in software maintenance. *IEEE Transactions on Software Engineering*, 34:407–432, May 2008.
- [13] P. Ellis. *The Essential Guide to Effect Sizes: Statistical Power, Meta-Analysis, and the Interpretation of Research Results*. Cambridge University Press, 2010.
- [14] A. Field and G. Hole. *How to Design and Report Experiments*. Sage publications Limited, 2003.
- [15] C. Gravino, G. Tortora, and G. Scanniello. An empirical investigation on the relation between analysis models and source code comprehension. In *Proceedings of the 2010 ACM Symposium on Applied Computing, SAC '10*, pages 2365–2366, New York, NY, USA, 2010. ACM.
- [16] O. M. Group. OMG Unified Modeling Language (OMG UML), Infrastructure, V2.1.2. Technical report, Nov. 2007.
- [17] J. Hannay and M. Jørgensen. The role of deliberate artificial design elements in software engineering experiments. *IEEE Transactions on Software Engineering*, 34:242–259, March 2008.
- [18] V. B. Kampenes, T. Dybå, J. E. Hannay, and D. I. K. Sjøberg. A systematic review of effect size in software engineering experiments. *Information & Software Technology*, 49(11-12):1073–1086, 2007.
- [19] A. N. Oppenheim. *Questionnaire Design, Interviewing and Attitude Measurement*. Pinter, London, 1992.
- [20] G. Scanniello, C. Gravino, and G. Tortora. Investigating the role of UML in the software modeling and maintenance - a preliminary industrial survey. In *Proceedings of the 12th International Conference on Enterprise Information Systems*, pages 141–148, 2010.
- [21] C. Wohlin, P. Runeson, M. Höst, M. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering - An Introduction*. Kluwer, 2000.